

Activity04: Computer Security

1. (Cryptanalysis) Though encryption is primarily designed to preserve confidentiality and integrity of data, the mechanism itself is vulnerable to brute force (statistical analysis). In other words, the more we see the encrypted data, the easier we can hack it. In this exercise, you are asked to crack the following cipher text. Please provide the decrypted result and explain your strategy in decrypting this text.

Cipher text

PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALOQSKR. QDFP FP ZK LIU
BROJZK MOLTROE.

a. Count the frequency of letters. List the top three most frequent characters.

P = 7 (11.9%)

R = 6 (10.2%)

O = 6 (10.2%)

F = 6 (10.2%)

b. Knowing that this is English, what are commonly used three-letter words and two-letter words. Does knowledge give you a hint on cracking the given text?

ลองดู word length: PRC⁸SOFQX = 8 FP = 2 QDR = 3 AFOPQ = 5 CZSPR = 5 LA = 2
JFPALOQSKR = 10 QDFP = 4 FP = 2 ZK = 2 LIU = 3 BROJZK = 6 MOLTROE = 7

คำที่น่าสนใจคือ FP, QDR, LA, QDFP, ZK, LIU

Common English 2-letter words เช่น of to In is it an as on at

Common 3-letter words เช่น the and for are but not you was

จุดที่สำคัญที่สุดคือ: FP ปรากฏหลายครั้ง และเป็นคำ 2 ตัวอักษร เราสามารถตั้งสมมติฐาน: FP = IS

จากนั้นลองกับ QDR ถ้า QDR = THE จะได้ mapping: Q -> T, D -> H, R -> E

ตอนนี้ลองคำแรก: PRC⁸SOFQX แทนตัวที่เรารู้: _ E _ _ _ I T _

เราจะเห็นว่าเป็นไปได้มากที่จะเป็น SECURITY จึงได้

$P \rightarrow S$

$R \rightarrow E$

$C \rightarrow C$

$S \rightarrow U$

$O \rightarrow R$

$F \rightarrow I$

$Q \rightarrow T$

$X \rightarrow Y$

ดังนั้น $PRCSOFQX \rightarrow SECURITY$, $FP \rightarrow IS$ และ $QDR \rightarrow THE$

c. Cracking the given text. Measure the time that you have taken to crack this message.

ลองแกะไปเรื่อย ๆ พบว่า

$P \rightarrow S$

$R \rightarrow E$

$C \rightarrow C$

$S \rightarrow U$

$O \rightarrow R$

$F \rightarrow I$

$Q \rightarrow T$

$X \rightarrow Y$

$A \rightarrow F$

$Z \rightarrow A$

$L \rightarrow O$

$J \rightarrow M$

$K \rightarrow N$

$D \rightarrow H$

$B \rightarrow G$

$I \rightarrow L$

$U \rightarrow D$

$M \rightarrow B$

$T \rightarrow T$

$E \rightarrow H$

ข้อความที่ crack ได้คือ

SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.

Take time about 30 mins.

d. Create a simple python program for cracking the Caesar cipher text using brute force attack. Explain the design and demonstrate your software.

(You may use an English dictionary for validating results.)

```
def caesar_decrypt(ciphertext, shift):
    result = ""

    for char in ciphertext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')

            decrypted = chr(
                (ord(char) - base - shift) % 26 + base
            )

            result += decrypted
        else:
            result += char

    return result

ciphertext = "KHOOR ZRUOG"

for shift in range(26):
    plaintext = caesar_decrypt(ciphertext, shift)

    print(f"Shift {shift:2}: {plaintext}")
```

✓ 0.0s

Shift 0:	KHOOR ZRUOG
Shift 1:	JGNNQ YQTNF
Shift 2:	IFMMP XPSME
Shift 3:	HELLO WORLD
Shift 4:	GDKKN VQKQC
Shift 5:	FCJJM UMPJB
Shift 6:	EBIIL TLOIA
Shift 7:	DAH#K SKNHZ
Shift 8:	CZGGJ RJMGY
Shift 9:	BYFFI QILFX
Shift 10:	AXEEH PHKEW
Shift 11:	ZWDDG OGJDV
Shift 12:	YVCCF NFICU
Shift 13:	XUBBE MEHBT
Shift 14:	WTAAD LDGAS
Shift 15:	VSZZC KCFZR
Shift 16:	URY#B JBEYQ
Shift 17:	TQXXA IADXP
Shift 18:	SP#WZ HZCWO
Shift 19:	ROWVY GYBVN
Shift 20:	QNUUX FXAUM
Shift 21:	PMTTW EWZTL
Shift 22:	OLSSV DVYSK
Shift 23:	NKRRU CUXRJ
Shift 24:	MJQQT BTWQI
Shift 25:	LIPPS ASVPH

โปรแกรมนี้จะลอง shift ไปเรื่อยๆ จนกว่าจะครบ 26 ตัวอักษร จากนั้นเราจะหาคำที่มีความหมาย
ดังตัวอย่างนี้ decrypt ได้ HELLO WORLD (shift = 3)

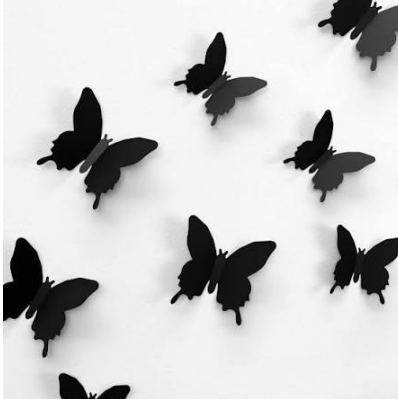
2. (Cryptanalysis on Symmetric Encryption) Vigenère is a complex version of the Caesar cipher. It is a polyalphabetic substitution.

a. Please review Kasiski examination Explain how it can be used to attack Vigenère.

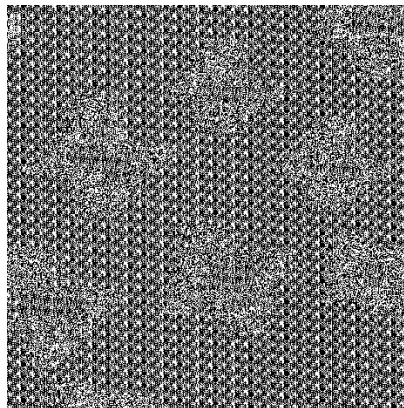
Kasiski examination attacks the Vigenère cipher by searching for repeated sequences in the ciphertext which is called “Superimposition”. When the same plaintext sequence is encrypted using the same positions of the repeating key, an identical ciphertext sequence may appear. The distances between repeated sequences are measured and factored to obtain possible key lengths. After estimating the key length, the ciphertext is divided into groups according to key position. Each group can then be analyzed as a Caesar cipher using frequency analysis. The recovered shifts form the Vigenère key, which can then be used to decrypt the ciphertext.

3. (Mode in Block Cipher) Block Cipher is designed to have more randomness in a block. However, an individual block still utilizes the same key. Thus, it is recommended to use a cipher mode with an initial vector, chaining or feedback between blocks. This exercise will show you the weakness of Electronic Code Book mode which does not include any initial vector, chaining or feedback.

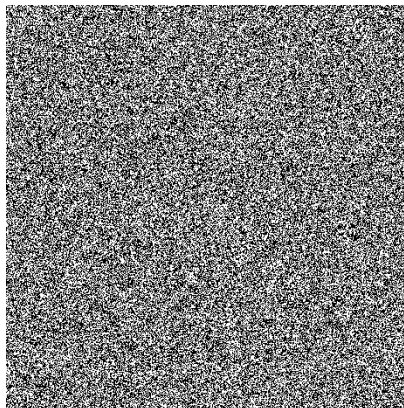
ORIGINAL



AES-256-ECB



AES-256-CBC



The experiment demonstrates that ECB mode leaks information about the structure of the plaintext because identical plaintext blocks are encrypted into identical ciphertext blocks. As a result, visible patterns in an image can remain even after encryption. CBC introduces chaining and an initialization vector, causing identical plaintext blocks to produce different ciphertext blocks in different positions. Therefore, a block cipher should generally use an appropriate mode of operation that prevents identical plaintext blocks from producing identical ciphertext blocks.

4. (Encryption Protocol - Digital Signature)

a. Measure the performance of a hash function (sha1), RC4, Blowfish and DSA. Outline your experimental design. (Please use OpenSSL for your measurement)

🔗 OpenSSL Speed 5 seconds

sha1

```
Doing sha1 for 5s on 16 size blocks: 26494898 sha1's in 4.89s
Doing sha1 for 5s on 64 size blocks: 21419676 sha1's in 4.95s
Doing sha1 for 5s on 256 size blocks: 10663013 sha1's in 4.81s
Doing sha1 for 5s on 1024 size blocks: 5597575 sha1's in 4.88s
Doing sha1 for 5s on 8192 size blocks: 960611 sha1's in 4.86s
Doing sha1 for 5s on 16384 size blocks: 431901 sha1's in 4.81s
```

RC4

```
Doing rc4 for 5s on 16 size blocks: 213206172 rc4's in 4.77s
Doing rc4 for 5s on 64 size blocks: 42193573 rc4's in 4.94s
Doing rc4 for 5s on 256 size blocks: 12537266 rc4's in 4.95s
Doing rc4 for 5s on 1024 size blocks: 3765580 rc4's in 4.73s
Doing rc4 for 5s on 8192 size blocks: 344217 rc4's in 4.92s
Doing rc4 for 5s on 16384 size blocks: 156651 rc4's in 5.00s
```

Blowfish

```
Doing blowfish for 5s on 16 size blocks: 51191023 blowfish's in 4.98s
Doing blowfish for 5s on 64 size blocks: 12040204 blowfish's in 4.97s
Doing blowfish for 5s on 256 size blocks: 3036034 blowfish's in 4.86s
Doing blowfish for 5s on 1024 size blocks: 553497 blowfish's in 4.78s
Doing blowfish for 5s on 8192 size blocks: 60481 blowfish's in 4.84s
Doing blowfish for 5s on 16384 size blocks: 29446 blowfish's in 4.95s
```

DSA

```
Doing 512 bits sign dsa's for 5s: 101127 512 bits DSA signs in 4.94s
Doing 512 bits verify dsa's for 5s: 166706 512 bits DSA verify in 4.95s
Doing 1024 bits sign dsa's for 5s: 55672 1024 bits DSA signs in 4.92s
Doing 1024 bits verify dsa's for 5s: 81213 1024 bits DSA verify in 5.00s
Doing 2048 bits sign dsa's for 5s: 21825 2048 bits DSA signs in 4.97s
Doing 2048 bits verify dsa's for 5s: 27388 2048 bits DSA verify in 4.97s
```

b. Comparing performance and security provided by each method.

Algorithm	Type	Performance	Security
SHA-1	Hash	ချာန OpenSSL	Weak / collision vulnerable
RC4	Stream cipher	ချာန OpenSSL	Broken / insecure
Blowfish	Symmetric block cipher	ချာန OpenSSL	Older design; generally not preferred for new systems
DSA	Digital signature	ချာန OpenSSL	Depends heavily on parameters and implementation

c. Explain the mechanism underlying Digital Signature. How does it combine the strength and weakness of each encryption scheme?

A **digital signature** is a cryptographic mechanism that provides authentication, integrity, and non-repudiation. First, the sender applies a **hash function** to the document to produce a fixed-size hash value, then uses their **private key** with asymmetric encryption to sign the hash, creating the digital signature. The receiver uses the sender's **public key** to verify the signature and compares the resulting hash with a newly calculated hash of the received document; if they match, the document is authentic and unchanged. **Hashing** is fast and efficient but cannot authenticate the sender. Symmetric **encryption** is fast for large amounts of data but has the weakness of secure key distribution, while **asymmetric encryption** provides authentication and easier key distribution but is slower. Therefore, digital signatures combine the strengths of hashing and asymmetric cryptography, and they sign only the **hash instead of the entire document** because asymmetric operations are computationally expensive, making this much more efficient for large documents.